cc

钢铁雄心—机器人对战平台

Hyper

Hyper 团队

目录

一、 引言 1

1.1 编写目的 1

1.2 整体背景 1

1.3 企业背景 1

1.4 编写说明 2

二、 目标与服务模型 2

2.1 需求分析 2

三、 技术路线与实现方案 2

3.1 技术栈 3

3.2 系统总体结构设计 4

3.3 数据库设计及其相关功能 5

3.4 接口总体设计 7

3.5 UI 设计 14

3.6 游戏玩法 16

3.7 解决方案 22

四、 项目亮点 30

4.1 基于 YOLOv8 的视觉传感电子围栏系统 30

4.2 Unity 实时战场模拟实况 32

4.3 基于 I^2C 协议的可弹出 Hyper 接口 33

五、 组织管理与开发过程 35

5.1 人员安排 35

5.2 时间安排 35

六、 物料清单与成本证明 37

七、 项目总结 43

一、引言

1.1 编写目的

本文档为融创软通的钢铁雄心 - 机器人对战平台使用及相关设计实现说明文档，为了更好地引导用户深入地了解与使用本产品。

1.2 整体背景

现在基于Arduino或树莓派开发板的小型组装自动避障车的产品很多，这些产品以完成预设功能为主，缺乏灵活性和拓展性。为打造一个既能保留基本避障能力，又可通过模块化扩展实现更丰富功能的新平台，本项目提出一种模块化兼容战斗车设计方案。该方案在现有自动避障车基础上，设置标准通用接口可快速安装各种外挂武器装备模块。所有模块实现可插拔设计，用户可以根据控制系统实现识别和管理各模块属性。需要自由配置不同组合。

除基础避障功能外，用户可通过扩展不同的攻击、防护等模块，打造出功能各异的战斗机器人，实现遥控对战。这不仅丰富了产品的应用范畴，也能增强用户的参与感和社交乐趣。此外，本设计提供开放接口与标准协议，鼓励开发者依据规范设计新型模块。模块生产可形成产业链，丰富平台应用生态，推动相关技术及爱好的发展。用户也可参与二次开发，获得更大满足感通过模块化和开放性设计，该方案实现了自动避障车的功能拓展与升级提供了一个可定制，社交性强，应用广泛的新型机器人平台。

1.3 企业背景

天津市融创软通科技股份有限公司(简称“融创软通”)，以“品质第一 •客户至上 •卓越执行”为理念，以矢志成为成为国内最专业的IT技术服务和人才服务提供商为核心价值观为客户提供更优质的服务。

服务范围包括：IT咨询及解决方案服务、应用开发及维护、软件产品工程、业务流程外包（BPO）服务、IT培训等。在金融、保险、电信、高科技、能源/公用事业等领域，具有深厚的行业积累和强大的技术服务能力。融创软通设立有大客户事业部、产品事业部、企业服务部，人才外包服务部，教育培训事业部，可以满足客户对于IT解决方案的咨询和项目实施、通用产品采购和产品合作研发、企业需求定制开发和人力资源外包的需求。

融创软通致力于拓展智慧、智能业务，具有智能SAAS运维、电商与众包、固定资产智能维保、智慧能源、智慧市政等为客户提供解决方案，同时在人工智能，大数据，区块链项目研发上有重大突破。

融创软通先后被评为“国家高新技术企业认证”，“雏鹰企业”，“ISO9001质量管理体系认证”、“CMMI3体系认证”等，同时又获得“天津市青年就业见习基地”、“国家级AAA信用认证企业”、“天津市软件行业协会会员单位”、“天津市产融合型企业”等多项荣誉。

融创软通系IT服务型高科技IT企业，通过多年的积累公司先后获得国家发明专利

1 项，软件产品著作权 23 项，支持教育部产学研协同育人项目近 40 个。截至目前公司已累计服务各类型企业 200 家以上，服务全国各高校超过 50 家以上，获得了合作伙伴们的一致好评。

1.4 编写说明

本文档将从以下几方面对本产品进行详细地阐述说明：

（1）目标与服务对象

（2）技术路线与实现方案

（3）项目亮点

（4）系统测试

（5）组织管理与开发过程

二、目标与服务模型

2.1 需求分析

2.1.1 基本功能

（1）车辆平台选型

（2）电子围栏方案

（3）外挂模块设计

（4）车辆控制

（5）数据通信

（6）模块挂载识别

（7）战斗车辆与用户的绑定

（8）服务端对用户信息的汇总

（9）排行榜机制和开发

（10）App 设计与开发

（11）对战规则设计

2.1.2 高级功能

引入摄像头并在 App 端展现第一视角。

三、技术路线与实现方案

3.1 技术栈

3.1.1 技术栈选择

在保证项目的开发效率以及产品质量的同时使用前沿 IT 技术，本项目决定使用如下图 1 所示的战斗车辆平台开发模式。在硬件方面，战斗车辆主板采用的是 Raspberry Pid 4B，并借助嘉立创 EDA 设计 PCB 电路板和主板。在软件方面，前端采用跨平台框架 Flutter，后端采用 Django 框架和 Rust 语言编写实时交互程序，并使用 MySQL 和 Redis 缓存数据库，3D 建模采用 SolidWorks ，并使用 Unity 进行游客 App 的构建，越界检测使用 YOLOv8 判断车辆是否超出电子围栏。

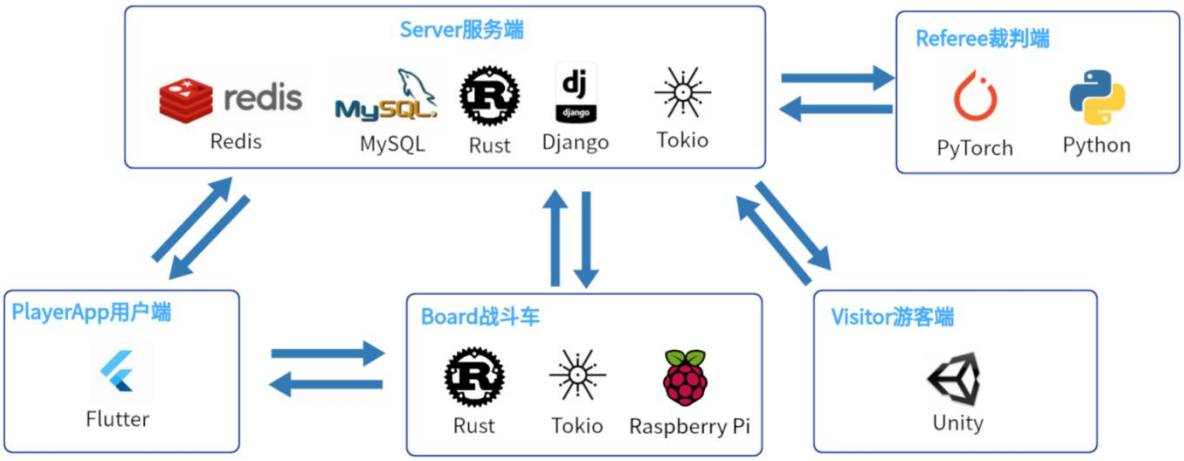

图 1 技术路线及技术实现方案

3.1.2 硬件开发技术

Raspberry Pi 4B 主板拥有丰富的外设接口和支持，可以轻松连接各种传感器、摄像头、显示器等外部设备，且因其具有丰富的文档和社区支持方便快速完成复杂需求，同时成本通常比较低，适合在预算有限的情况下进行项目开发。

基于此，Raspberry Pi 4B 主板高度符合了本项目作为模块化兼容战斗车，需支持模块化与开放性设计并引入摄像头以在 APP 端展现第一视角的需求，因而本团队选择 Raspberry Pi 4B 作为开发主板。

3.1.3 手机战斗车控制端 app 开发技术

前端开发框架本团队采用 Flutter。其作为前端开发框架可以带来跨平台支持、快速开发、自定义能力强、良好的性能和强大的社区支持等优势，非常适合用于控制手机端操作战斗小车的应用场景

3.1.4 实时交互服务端(websocket)开发技术

由于需要实时获取战斗车的位置，为了满足低延迟的需求，本团队将服务端分为**性**

能敏感和**性能不敏感**的部分。

对于**性能敏感**部分，本团队使用 Rust 和 WebSocket 协议，并设计了一套简洁的交互接口交换状态信息。Rust 编译器生成的代码与 C/C++相媲美，拥有出色的性能；WebSocket 协议支持全双工通信，同时作为一种轻量级、高效的通信协议，能够实现更快速的数据传输和更低的延迟，这都有助于保证系统的响应速度和稳定性。

对于**性能不敏感**部分，本团队使用 Django 框架以及 HTTPS 协议来完成快速迭代开发。

数据库服务则使用 MySQL 和 Redis 分别用于长期存储和短期低延时响应存储。

3.1.5 建模技术

本团队使用 Solidworks 作为计算机辅助设计软件完成建模设计。

3.1.6 电子围栏技术

本团队使用 YOLOv8 进行实时位置扫描判断出界。YOLO 是一种基于深度学习的实时目标检测算法，而 YOLOv8 是其第八个版本。其实时性和多目标检测能力能够时刻判断战斗车位置，且其具有的高度准确性和稳定性支持在激烈的战斗中灵敏判断出界情况。其效果如图 2 所示。

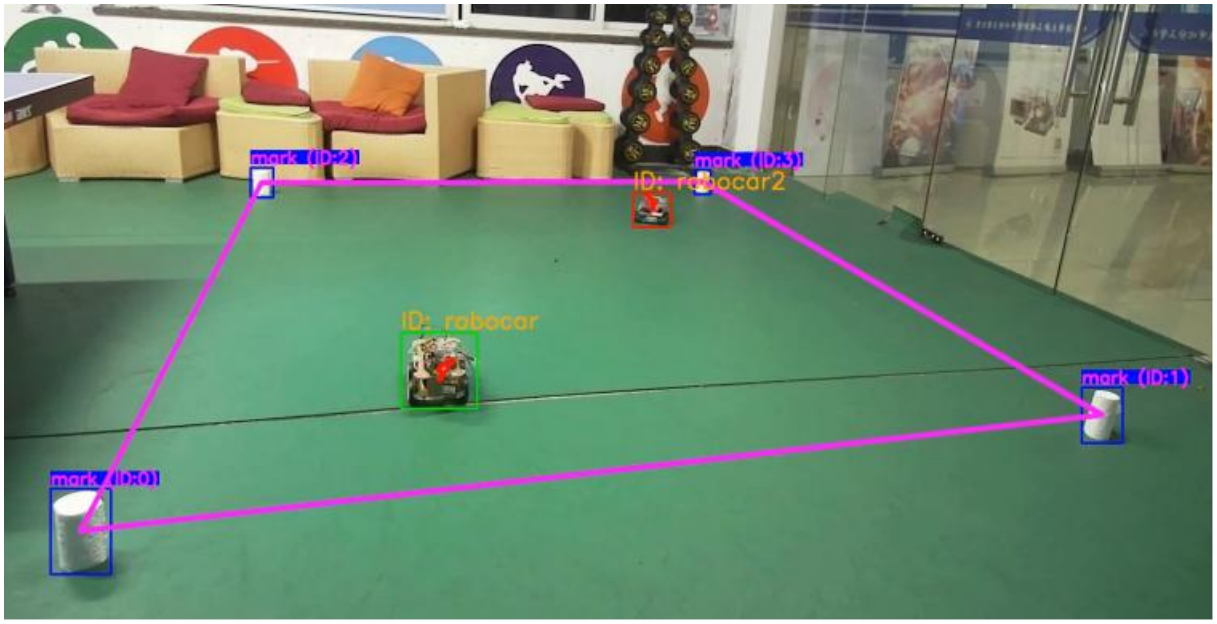

图 2 电子围栏

3.2 系统总体结构设计

根据 2.1 中的需求分析，本项目将对用户提供如图 3 所示功能。

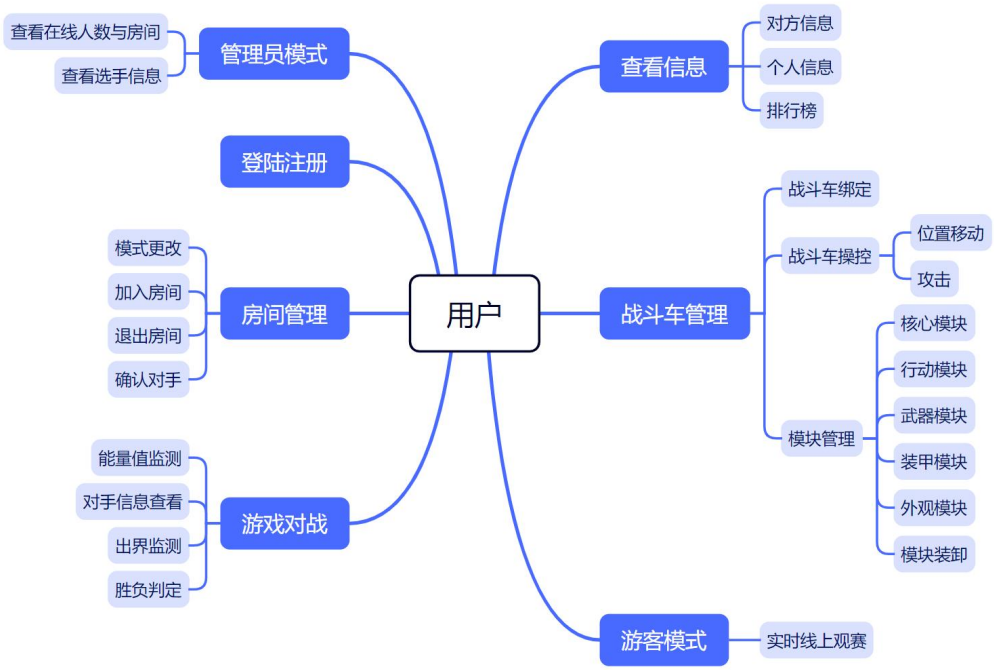

图 3 系统总体架构

3.3 数据库设计及其相关功能

任何系统都离不开数据的处理，数据处理的基础就是数据库的设计与应用技术。为了实现系统中的功能，团队建立了独立的数据库，在一定程度下保证了用户信息的安全。以下列举了部分数据库设计。

3.3.1 用户绑定战斗车

在绑定过程中，一名用户只能绑定一台战斗车，属于一对一的关系。如图 4 所示。

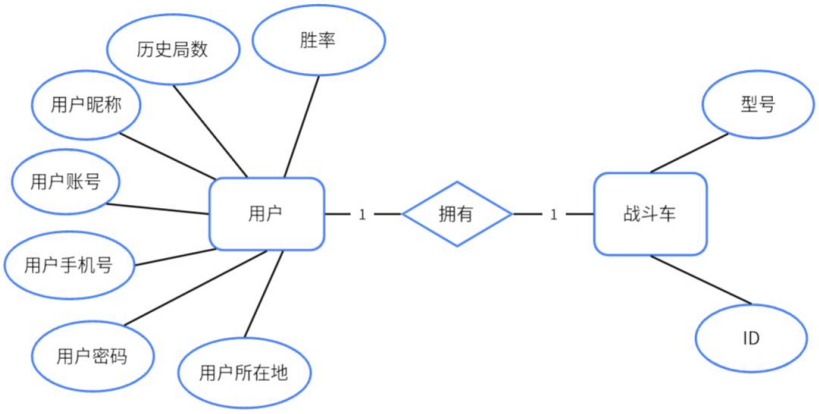

图 4 用户绑定战斗车 E-R 图

3.3.2 用户战绩

在用户战绩记录中，一个用户每结束一局对战就会增加一次战绩，属于一对多的关系。如图 5 所示。

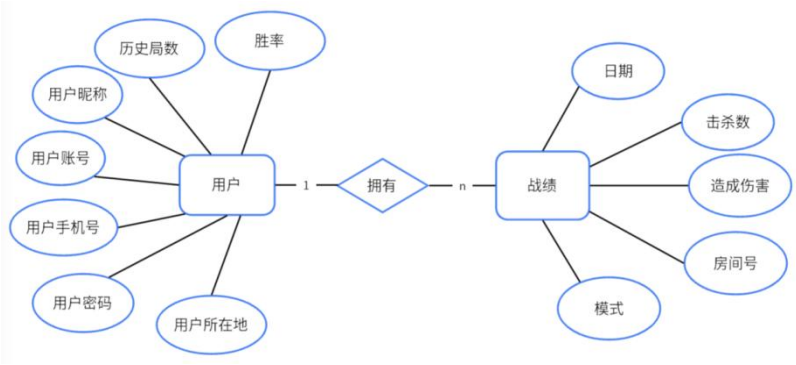

图 5 用户战绩 E-R 图

3.3.3 用户信息表

字段名	类型	说明
uaccount(主键)	CHAR(32)	用户账号
upassword	VARCHAR(45)	用户密码
uphone	VARCHAR(30)	用户手机号
history_num	INT	对战局数
win_rate	DOUBLE	胜率
uname	VARCHAR(200)	用户昵称
uphone	VARCHAR(30)	用户手机号码

3.3.4 战车信息表

字段名	类型	说明

id(主键)	CHAR(32)	战斗车 ID
model_type	VARCHAR(50)	战斗车型号

3.3.5 对战记录信息表

字段名	类型	说明
id(主键，自增)	CHAR(32)	本次对战记录 ID
date_time	DATE	对战日期与时间
room_id	CHAR(32)	房间号
battle_mode	VARCHAR(10)	对战模式
kill_count	INT	摧毁敌方战车数量
damage	INT	造成伤害

3.4 接口总体设计

3.4.1 接口总体思路

由于本团队把服务端切分成两个部分，下面分别针对 HTTPS(Django)部分和 Websocket(Rust Tokio)部分分别进行阐述。

对于 HTTPS 部分，数据传输格式使用 JSON 格式，使用 CBV（class based view）混合 FBV（function based view）以完成接口实现。由于 Django 本身就对数据库进行了一次封装，掩盖了所有 SQL 语句，故不存在 SQL 注入式攻击的可能，并且以此简化了对关系模型的迭代并支持回滚。此外通过 Django 的 RESTFUL 中间件可以完成大部分简单接口的设计，例如使用 ModelViewSet 可以直接生成满足 RESTFUL 规范的接口，并可以通过继承机制进行修改。

对于 Websocket 部分，由于低延迟需求，数据传输格式使用我们自定义的领域特定格式(Domain Specific Format)，使用 Rust 广泛使用的 Tokio 异步运行时来完成网络连接以及信息交换，并将连接对象分为四个角色 Board、PlayerApp、Visitor、Referee 分别对应 战车、用户控制 App、实时战场虚拟观战（详见 4.2）的游客、YOLOv8 裁判端

（参考 4.1）。

3.4.2 主要接口展示

（1）Websocket 接口

本团队将交互分为五个角色：Board, PlayerApp, Server, Visitor,Referee

①Server <=> Board

由 Board 发起：

（a）register_board:($lan_ip,$board_name,$wifi_name)

功能：向服务端声明一个战车（未连接玩家 PlayerApp）

参数：

字段名	必选	说明
lan_ip	是	战车所在局域网中的 ip
board_name	是	战车的 linux 主机名
wifi_name	是	战车连接 wifi 名

返回值：response_register_board:($is_successful)

参数：

字段名	必选	说明
Is_successful	是	操作是否成功

由 Server 发起：

request_check:()

功能：要求 Board 立即发送 check:() 以判断是否断线

返回值：check:()

参数：无

②Server <=> PlayerApp

由 PlayerApp 发起：

(a)register_app:($user_name,$room_num,$board_id)

功能：向服务端声明登录玩家 App

参数：

字段名	必选	说明
user_name	是	玩家用户名
room_num	是	打算加入的房间号
board_id	是	所选战斗车的 id 号

返回值：response_register_app:($is_successful)

参数：

字段名	必选	说明
Is_successful	是	操作是否成功

（b）request_list_rooms:()

功能：向服务器声明获取当前房间列表

参数：无

返回值：

```
reponse_list_rooms([
    (boardname1, wifi_name, lan_ip),
    (boardname2...)
    ...
])
```

（c）request_list_boards:()

功能：向服务器声明获取当前战斗车列表

参数：无

返回值：

response_list_board([

(boardname1),

(boardname2),

...

])

（d）request_set_board_coords:($border_id,$x,$y)

功能：向服务器声明获取当前战斗车参数

参数：

字段名	必选	说明
border_ip	是	战斗车 id
x	是	战斗车的横坐标
y	是	战斗车的纵坐标

返回值：

request_set_board_coords([

(boardname1,x,y),

(boardname2,x,y)

...

])

由 Server 发起：

reponse_check:()

功能：要求 Board 立即发送 check:() 以判断是否断线

返回值：check:()

参数：无

③Server <=> Referee

由 Referee 发起：

register_referee:($room_num)

功能：向服务端声明进入该房间

参数：

字段名	必选	说明
room_num	是	打算加入的房间号

返回值：response_register_app:($is_successful)

参数：

字段名	必选	说明
Is_successful	是	操作是否成功

（b）request_list_rooms:()

功能：向服务器声明获取当前房间列表

参数：无

返回值：

```
reponse_list_rooms([
    (boardname1, wifi_name, lan_ip),
    (boardname2...)
    ...
])
```

（c）request_list_boards:()

功能：向服务器声明获取当前战斗车列表

参数：无

返回值：

```
response_list_board([
    (boardname1),
    (boardname2),
    ...
])
```

（d）request_list_boards_in_room:($room_num)

功能：向服务器声明获取指定房间号的战斗车

参数：

字段名	必选	说明
room_num	是	房间号

返回值：

```
reponse_list_boards_in_room([
    (boardname1),
    (boardname2)
    ...
])
```

（d）request_set_board_coords:($border_id,$x,$y)

功能：向服务器声明获取当前战斗车参数

参数：

字段名	必选	说明
border_ip	是	战斗车 id
x	是	战斗车的横坐标
y	是	战斗车的纵坐标

返回值：

```
reponse_set_board_coords([
    (boardname1,x,y),
    (boardname2,x,y)
    ...
])
```

由 Server 发起：

request_check:()

功能：要求 Board 立即发送 check:()以判断是否断线

返回值：check:()

参数:无

④Server <=> Visitor

由 Visitor 发起：

（a）register_visitor:($room_num)

功能：向服务端声明进入该房间

参数：

字段名	必选	说明
room_num	是	打算加入的房间号

返回值：response_register_app:($is_successful)

参数：

字段名	必选	说明
Is_successful	是	操作是否成功

（b）request_list_boards_in_room:($room_num)

功能：向服务器声明获取指定房间号的战斗车

参数：

字段名	必选	说明
room_num	是	房间号

返回值：

request_list_boards_in_room([

(boardname1),

(boardname2)

...

])

（b）request_get_model_type:()

功能：向服务器声明获取当前房间号的战斗车

参数：无

返回值：response_get_model_type:($model_type)

参数：

字段名	必选	说明
model_type	是	返回战斗车型号

由 Server 发起：

request_check:()

功能：要求 Board 立即发送 check:() 以判断是否断线

返回值：check:()

参数：无

3.5 UI 设计

3.5.1 页面风格

本项目采取扁平化设计风格，以强调简约、干净、直观的设计，去除了繁琐的效果和装饰，突出了内容和功能的重要性著称。

采用扁平化设计，让用户能够轻松理解与使用软件，通过去除多余的设计元素和效果，帮助用户更快找到他们所需的信息与功能。

扁平化设计通常使用简单的图形与颜色，这可以加快软件的加载速度，让用户体验更上一层楼。

扁平化设计在适配不同的设备、屏幕尺寸大有优势，可以平衡用户界面与用户体验之间的平衡关系。

扁平化设计作为一种现代化时尚设计，能够很好地组织内容，使整体界面更加整洁，内容更清晰明了，用户也能对所需信息一目了然，是本软件设计风格的不二之选。如图

6 为扁平化设计的应用以及展示。

图 6 扁平化设计应用展示

3.5.2 交互设计

在移动应用设计中，交互效果的巧妙使用，在提升用户感知度和感官体验上可以起到无可替代的作用，如下：

（1）元素颜色变化：

在本项目中，当用户点击相应按钮进行交互或者跳转时，相应的按钮会更换颜色，以提醒用户所点的内容。这既能保证页面的的整洁也能帮助用户对应用的布局有更深的了解。

（2）提示框：

在本项目中，当用户每次进行录入或是修改等操作时，应用都会在操作结束后以提示框的形式告知用户其操作成功与否，如未成功，则会告知原因。这些提示不仅在使用户拥有更优良的体验，也暗示与鼓励用户更多地尝试软件的相关功能。

（3）位置移动：

在本项目中，在操控战斗车移动时，方向键的中心会随手指移动而移动，以帮助用户确定当前战斗车前进方向。这既能加强玩家和战斗车之间的交互性，也能帮助玩家更加灵巧方便的操控战斗车。

3.6 游戏玩法

3.6.1 属性值

战斗车包含四项基本属性值，由行动模块的类型决定战斗车基本属性：

（1）能量值：即为“血量”，每次被敌方武器模块攻击到，就扣除相应模块的能量值。能量值的大小由所挂载模块的类型决定。

①当战斗车的装甲模块的能量值降为 0 时，装甲模块将会脱落；

②当武器模块的能量值降为 0，武器模块将会脱落，战斗车无法再攻击；

③当行动模块的能量值降为 0，行动模块将无法运作，战斗车将无法移动；

④当核心模块的能量值降为 0，即战斗车被“摧毁”，无法再运行。

（2）攻击力：为战斗车武器模块一次攻击所能扣除对方能量值的量。攻击力的大小由所挂载模块的类型决定。

（3）动力值：为战斗车的移动速度。动力值越大，战斗车移动速度越快；动力值越小，移动速度越慢；若行动模块能量值降为 0，则动力值降为 0，无法移动。动力值的大小由战斗车总重量决定，总重量越重，动力值越小；总重量越轻，动力值越大。

（4）总重量：为战斗车各个挂载模块的重量之和。总重量影响动力值，总重量越大，战斗车动力值越小，移动速度越慢；总重量越小，战斗车动力值越大移动速度越快。总重量的大小由战斗车各个挂载模块决定。

3.6.2 外挂模块设计

四轮式战斗车全貌如图 7 所示。

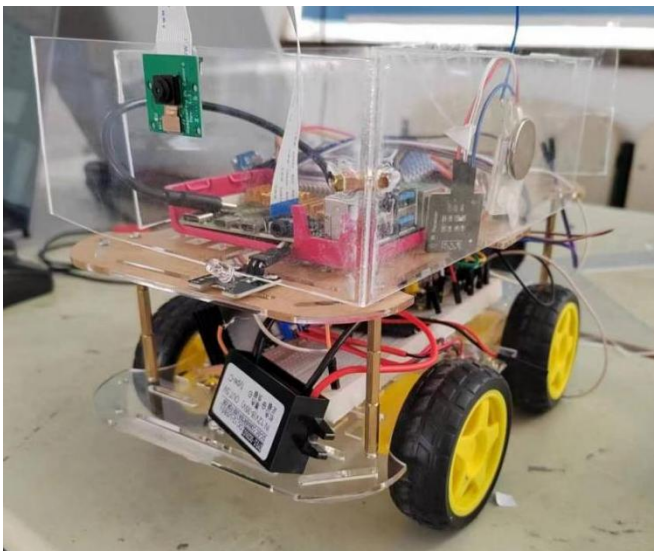

图 7 四轮战斗车

（1）核心模块

核心模块位于战斗车中央，被多个装甲模块所包围“保护”，为战斗车的“心脏”。若核心模块能量值降为 0，则战斗车瞬间“损毁”，无法再运作。例样如图 8 所示。

包含属性值：能量值

能量值范围：100

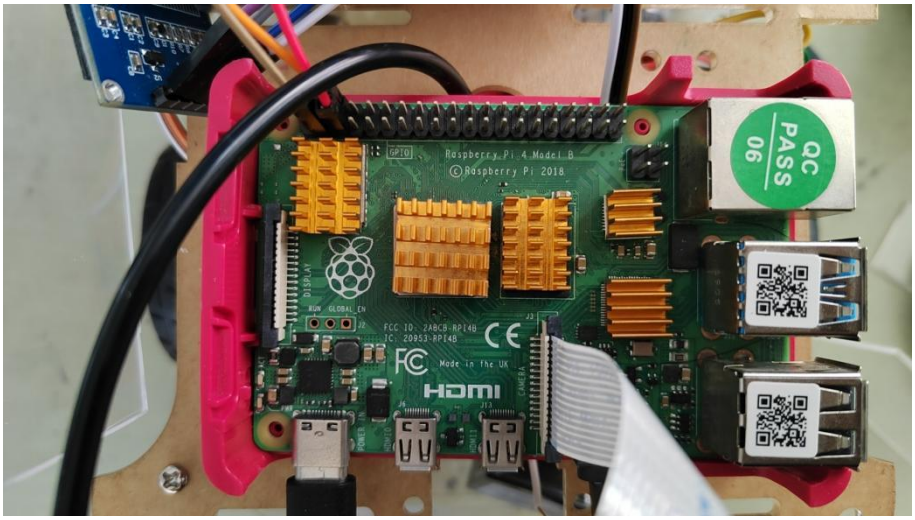

图 8 核心模块

（2）装甲模块

装甲模块位于战斗车外表，为战斗车的“护盾”。一辆战斗车往往装有多个装甲模块。装甲模块能量值较高，往往能承受多次攻击。若其中一块装甲模块能量值降为 0，则该装甲模块脱落，不再起到防护作用。例样如图 9 所示。

包含属性值：能量值、重量

能量值范围（与装甲模块的重量成正比）：10-50

重量范围（与装甲模块的能量值成正比）：10-50

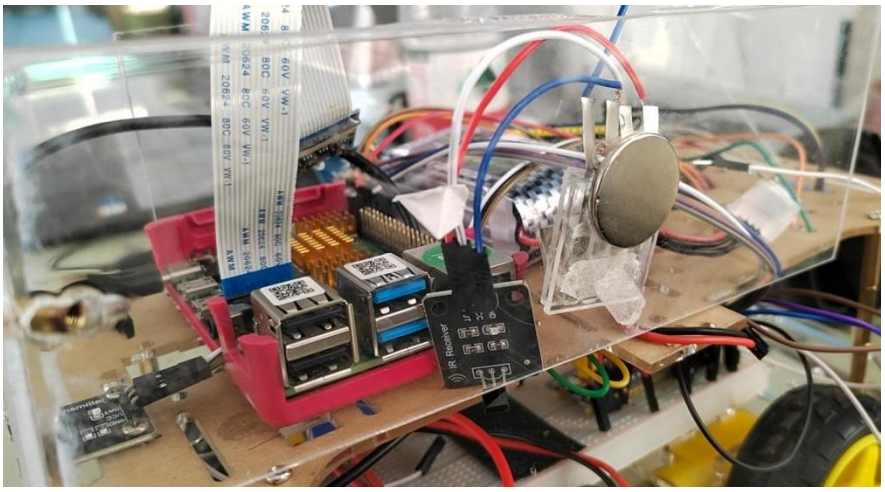

图 9 装甲模块

（3）武器模块

武器模块位于战斗车表面，为战斗车的攻击武器。武器模块能够发射红外线，即为“发射子弹”。一辆战斗车只能装有一件武器。若武器模块能量值降为 0，则武器模块脱落，无法再进行攻击。

其中武器模块又分为固定武器和可旋转武器。固定武器即攻击方向固定，只能朝着战斗车行进方向攻击，灵活性较差，但重量较轻；可旋转武器可以单独旋转攻击模块的朝向，改变攻击方向，灵活性较强，但重量较重。固定武器如图 10 所示，可旋转武器如图 11 所示。

包含属性值：攻击力、能量值、重量

攻击力范围（与武器模块的重量成反比）：10-50

能量值范围：10

重量范围（与武器模块的攻击力成反比）：10-50（固定武器）、30-80（可旋转武器）

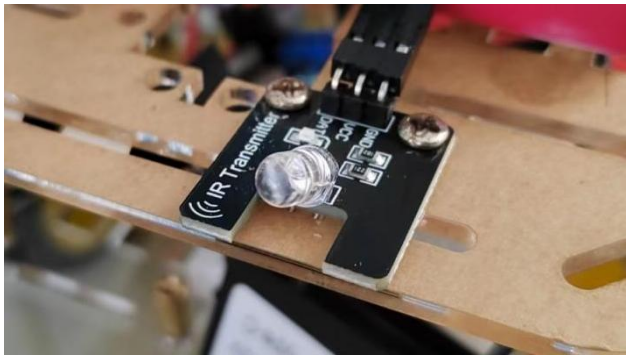

图 10 固定武器

图 11 可旋转武器

（4）行动模块

行动模块位于战斗车底部，为战斗车的“腿脚”。行动模块的选择对战斗车的基本属性起到决定性作用。

行动模块分为履带式和四轮式。履带式的特点是笨重、防御高、火力强，四轮式的

特点是轻快、防御低、火力弱。

行动模块会为战斗车提供基础速度值，为武器模块提供攻击力与能量值加成，为装甲模块提供能量值加成。

当行动模块能量值降为 0，则行动模块无法再运作，需要等待 5 秒方可再次运作。

四轮式如图 12 所示，履带式如图 13 所示。

包含属性值：动力值

基础动力值：40（四轮式）、35（履带式）

攻击力加成：+5（四轮式）、+8（履带式）

能量值加成：+4（四轮式与固定武器）、+2（四轮与可旋转武器）、+8（履带式与固定武器）、+6（履带式与可旋转武器）

图 12 四轮式

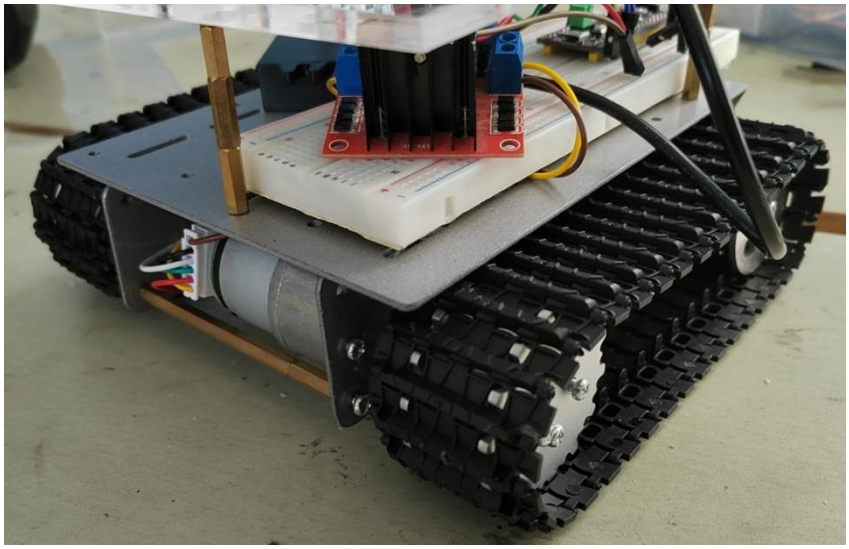

图 13 履带式

（5）外观模块

外观模块位于战斗车外表。起到美观作用。

包含属性：重量

重量范围：10-50

（5）第一视角摄像头

第一视角摄像头位于战斗车外表前方，可便于玩家更方便操作与查看。

第一视角摄像头如图 14 所示，其效果如图 15 所示。

包含属性：无

图 14 第一视角摄像头

图 15 第一视角摄像头效果

3.6.3 组装原理图

组装原理图如图 16 所示。

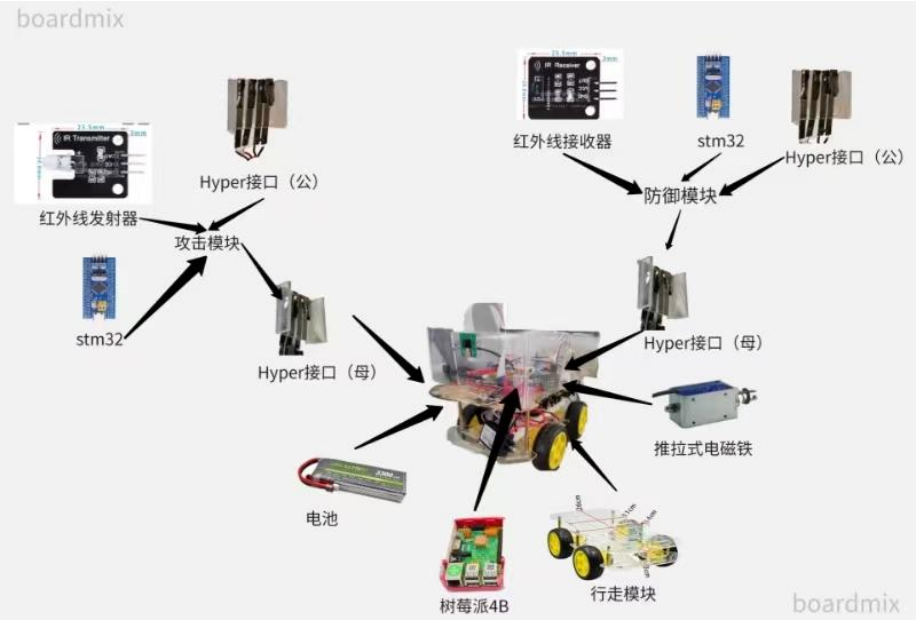

图 16 组装原理图

3.6.4 对战规则

（1）登录与战斗车选择

游戏开始前需注册登陆账号，登陆后需借助 WiFi 链接到战斗车，即可进入游戏。每个账号只能绑定一辆战斗车。

（2）房间的创建与对战对战的选择

对战模式有两种，分别为两队对抗与混战。

两队对抗模式下，每队需 4 名玩家，两队人数都满足 4 人后，即可由房主选择开始对战。该模式下，两队玩家自由操作，以在游戏时限内“摧毁”敌方所有战斗车为目标。

混战模式下，房间内人数需满足 2-8 人，人数满足即可由房主选择开始对战。该模式下，房间内所有玩家自由操作，以尽可能多的“摧毁”其余玩家战斗车为目标。

（3）单局时长

游戏默认时常为 15 分钟。

（4）胜负判定

①两队对抗模式

在游戏时限内，若一队中再无可作战的战斗车（即所有战斗车全部“损毁”），则该队判为输方，另一队判为胜方。

若游戏时间结束两队仍有可作战的战斗车，则根据可作战的战斗车数量评定胜负，数量多的一方判为胜方，数量少的一方判为败方；

若数量一样，则剩余核心模块总血量多的一方判为胜方，总血量少的一方判为败方；

若数量一样的同时剩余核心模块总血量也一样，则判为平局。

②混战模式

在游戏时限内，若场上剩余可作战的战斗车仅余一辆，则游戏结束；

若游戏时间结束，场上依旧有可作战的战斗车，则游戏结束。

（5）单局结束积分结算

①两队对抗模式

胜方所有队员积分均+2，败方所有队员积分均-1。

②混战模式

游戏结束后会根据每个玩家在本局游戏中摧毁的他人战斗车数量以及攻击他人战斗车造成的综合伤害（对装甲模块造成的伤害*0.8+对核心模块造成的伤害*1.1）进行综合评分，最后根据评分排名，前一半排名玩家增加积分，后一半排名玩家扣除积分（若参赛人数为奇数，则前一半人数向上取整）。

积分增加形式为前一半排名最后一名玩家积分+1，再上面一名玩家积分+2，以此类推（例如参加人数为 8 人，则第四名积分+1，第三名积分+2，以此类推；参赛人数为 5 人，则第三名积分+1，第二名积分+2，以此类推）。

积分扣除形式为后一半排名第一名玩家积分-1，再下面一名玩家积分-2，以此类推（例如参加人数为 8 人，则第五名积分-1，第六名积分-2，以此类推；参赛人数为 5 人，则第四名积分-1，第五名积分-2）。

（6）作弊检测

每局游戏结束后都会记录本局游戏对战数据，包括每辆战斗车的行进路程和攻击次数，基于此，对玩家是否存在作弊行为进行检测，检测方法详见 3.7.9。

3.7 解决方案

本产品为解决操控战斗车与查看信息的难题，采用分治的思想将大问题微小化，将整个系统抽象为多个具体功能模块，实现项目所需的所有功能。

3.7.1 登陆与注册

登录与注册是使用一个线上软件的基础，任何软件都离不开登录与注册。合理的登录注册功能可以优化用户使用体验，与图 17 和图 18 分别为登录与注册的流程图。

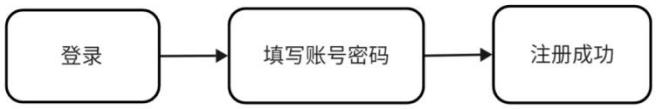

图 17 登录流程

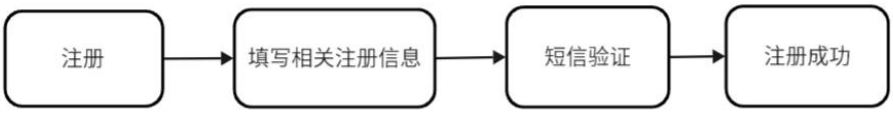

图 18 注册流程

3.7.2 战斗车选择

需先打开战斗车上的信号开关，然后打开手机蓝牙，待登陆成功后，即可选择对应战斗车。图 19 为战斗车选择流程图。

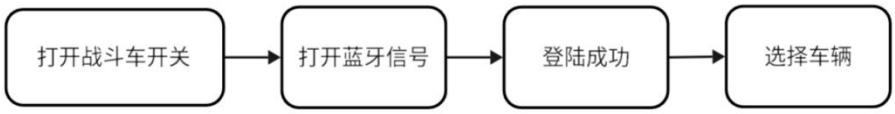

图 19 战斗车选择流程

3.7.3 房间准备

在房间准备页面，本团队设计了三大模块：房间内玩家状态模块、房间状态模块、模块连接状态。如图 20 为房间准备页面。

图 20 房间准备页面

下面本团队将具体介绍每个模块涉及的功能：

（1）房间内玩家状态模块

房间内玩家状态模块占据屏幕左边及中央大部分位置，列向显示着房间内每一位玩家的昵称与当前准备状态。在每一行中左侧是玩家的昵称，右侧是玩家当前准备状态。准备状态分三种情况，为“已准备”“未准备”“房主”，分别对应着图形“绿色勾”“红色叉”“黄色星”。在状态改变的同时也会改变左侧昵称的颜色显示，绿色名字对应“已准备”，红色名字对应“未准备”，黄色名字对应“房主”。如图 21 所示。

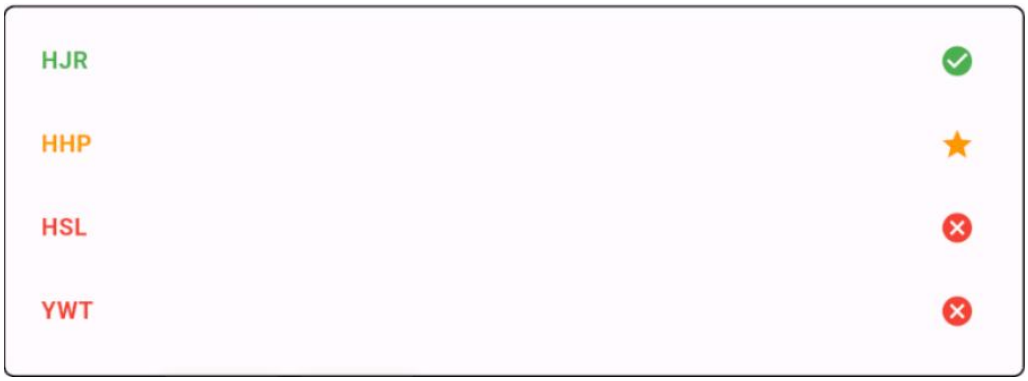

图 21 房间内玩家状态模块

（2）房间状态模块

房间状态模块位于页面的右边，列向显示着房间信息与选项，其中可查看当前房间的游戏模式；选项中包括“游戏模式选择”“开始游戏”“退出房间”，其中“游戏模式选择”和“开始游戏”只能由房主来操作。如图 22 所示。

图 22 房间状态模块

（3）模块连接状态模块

模块连接状态模块位于页面底部，显示当前玩家的战斗车以挂载的可用模块。如图 23 所示。

图 23 模块连接状态模块

3.7.4 遥控战斗车对决

在遥控战斗车页面，包含四大模块：移动模块、攻击模块、剩余能量值模块、敌方信息模块。如图 24 为遥控战斗车页面。

图 24 遥控战斗车页面

下面本团队将具体介绍每个模块涉及的功能：

（1）移动模块

移动模块位于页面左侧，只需手指拖动中央紫色按钮，便可让战斗车做出相应方向移动。如图 25 所示。

图 25 移动模块

（2）攻击模块

攻击模块位于页面右端，只需点击攻击按钮，战斗车便会进行攻击（设射击）。如图 26 所示。

图 26 攻击按钮

（3）剩余能量值模块

剩余能量值模块位于页面左上角。有三条分别为绿色、橙色、红色的长条，其中绿色代表当前核心模块的剩余能量值，橙色代表当前装甲模块的剩余能量值，红色代表当前武器模块的剩余能量值。如图 27 剩余能量值模块。

图 27 剩余能量值模块

（4）敌方信息模块

敌方信息模块位于页面右上角。默认情况下该模块会被隐藏，若需查看，只需点击

右上角小三角形，即可显示当前房间内剩余玩家的剩余能量值。如图 28 所示。

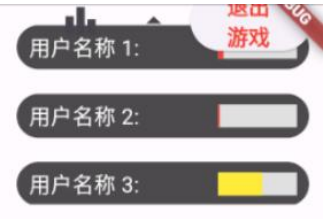

图 28 敌方信息模块

其中在当前页面的上面中央，显示着本局游戏的剩余时间，当剩余时间不足 1 分钟，时间颜色将变为红色，如图 29 所示。

图 29 剩余时间

3.7.5 查看信息

（1）确认对战对手与玩家信息

在房间准备页面，点击其余玩家，即可查看其余玩家个人信息，查看信息包括用户名、地区、对战局数、胜率、历史战绩以及各项能力值。帮助玩家更好的了解作战对手的信息或是队友的信息以更好地对战或配合，如图 30 所示。

图 30 玩家个人信息

（2）查看排行榜单

在房间准备页面，点击右上角图标，即可查看当前排行榜。

排行榜分为三类，分为总榜、月榜、地区周榜。其中总榜实时刷新，显示当前玩家总积分排名，月榜实时显示当前月玩家所获积分排名，地区周榜实时显示本周本地区玩家所获积分排名。

此外为了强调排名靠前玩家的成就与表现，也为了激励其他玩家，本团队对排行榜前三的玩家做了突出显示，以彰显其非凡的成就与表现。如图 31 所示。

图 31 排行榜

3.7.6 模块挂载与重量检测计算

模块可自由安插在 I^2C 接口上也可自由卸载拔出。主板会定时动态扫描所有可用的

I^2C 接口，判断是否是由可用模块来完成模块的挂载与卸载，并计算当前车身重量。若挂载模块为非可用模块，则发出警报；若车身重量超出平台规定，则发出警报。

3.7.7 模块弹出

本项目选用电磁铁作为弹出的执行器。当模块能力值为零时，电磁铁将释放模块，同时将连线断开,即表示模块已损毁或失效。在这里本项目使用了 Hyper 接口(详见 4.3)来完成弹出这一功能。

3.7.8 胜负裁定

详细规则请见 3.6.4。

如图 32 所示，即为混战模式结算页面。

图 32 混战模式结算页面

3.7.9 作弊检测

针对作弊问题，本团队提出“战斗烈度”这一概念作为指标以判断本局对战是否存在作弊行为。

“战斗烈度”，包括一局对战中战斗车每分钟的总移动距离与攻击操作次数。根据“战斗烈度”是否偏离系统的预设值，从而判断是否存在作弊行为。

系统的预设值的计算与设置本团队通过使用均值加减标准差的倍数得出。

具体计算方式以移动距离为例：

（1）设有一组数据 $S=\{s_1,s_2,...,s_n\}$ 表示战斗车每分钟的行进路程，s_n 为第 n 分钟的行进路程。

（2）计算 S 的均值 $\bar{s}$ 和标准差 σ

$$\bar{s}=\frac{1}{n}\sum_{i=1}^{n}s_i \qquad \sigma=\sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(s_i-\bar{s}\right)^2}$$

（3）根据均值和标准差，对于均值加减 k 倍标准差，计算预设值

上限：$\bar{s}+k\times\sigma$

下限：$\bar{s}-k\times\sigma$

本团队根据大量游戏数据，选择将 k 定为 2，将“战斗烈度”中的每分钟移动距离定为 0.2-0.4 米/秒，攻击操作次数定为 7-13 次/秒。

3.7.10 服务端信息汇总

在服务端，可以查看当前在线用户人数和当前已创建房间数。如图 33 和图 34 所示。

图 33 服务端管理员界面

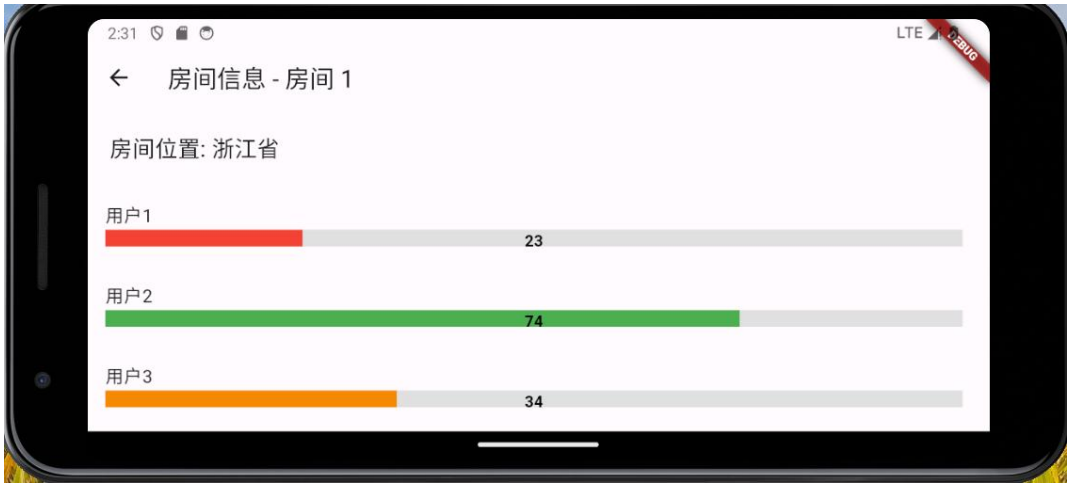

图 34 当前房间信息界面

四、项目亮点

4.1 基于 YOLOv8 的视觉传感电子围栏系统

为实现电子围栏与战斗车出界监测判断，本团队采用 YOLOv8 作为目标检测算法，ByteTrack 作为目标追踪算法，使得模型在保证轻量化的程度上准确识别并且追踪战斗车。

在 YOLOv8 识别获取战斗车与电子围栏柱目标后，将电子围栏柱连接形成多边形电子围栏区域，再使用射线法判断战斗车的是否在多边形内，从而判断战斗车出界情况

模型一：YOLOv8

YOLOv8（You Only Look Once version 8）是一种先进的目标检测模型。

其主要思路是均匀地在图片的不同位置进行密集抽样，抽样时可以采用不同尺度和长宽比，然后利用相关联的卷积神经单元多次提取特征后直接进行分类与回归，整个过程只需要一步，所以其速度较快。其模型结构如图 35 所示。

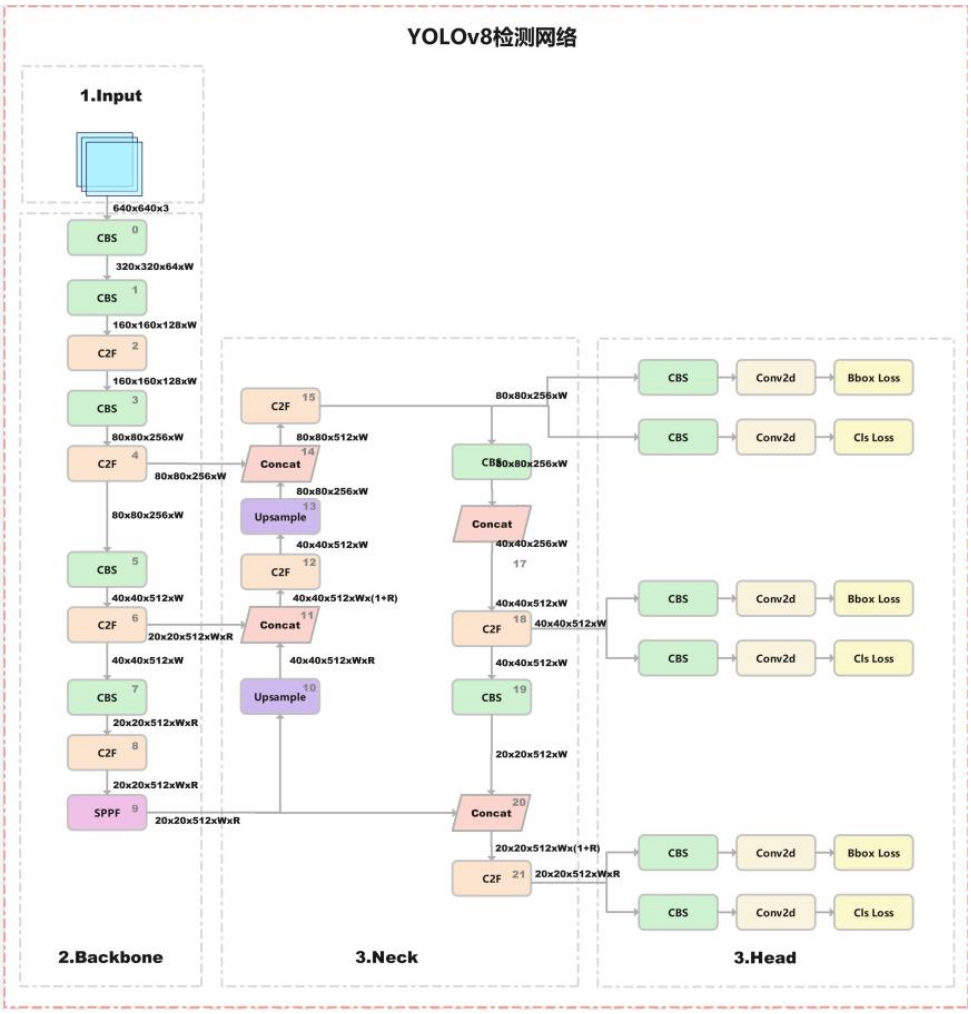

图 35 YOLOv8 模型结构

本任务中，通过 yolov8 可以很快的判别出战斗车位置，交由相关接口进一步操作。

模型二：ByteTrack

ByteTrack 是基于 tracking-by-detection 范式的多跟踪方法，发布于 2022 年的顶会期刊，相较于其余追踪算法有很大的提升。追踪算法性能对比如图 36 所示。

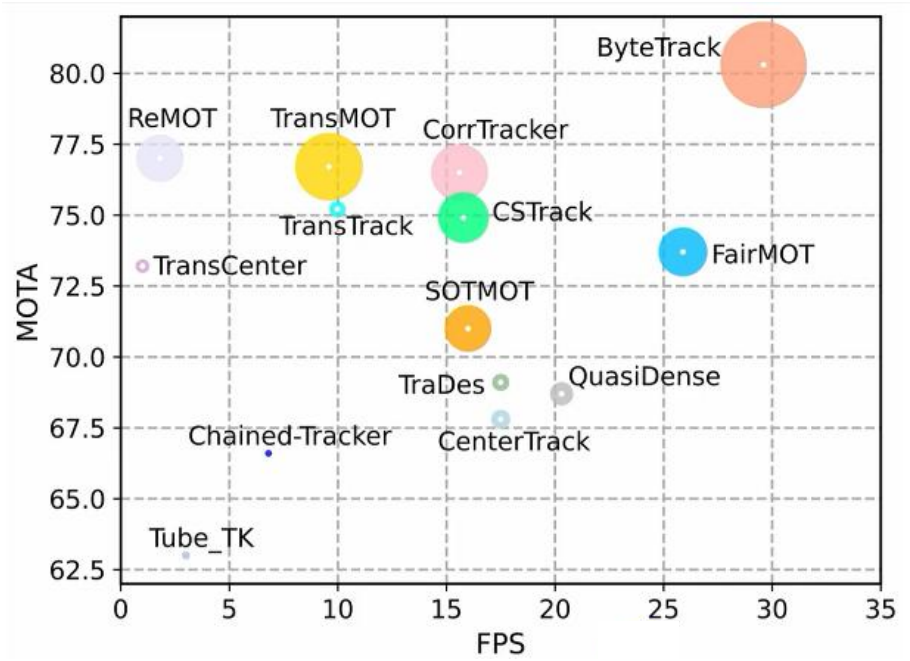

图 36 追踪算法性能对比

ByteTrack 旨在针对视频帧画面中感兴趣或想要追踪的目标进行检测并获取在图像中的位置，且对每个目标分配一个 ID，在目标运动过程中，维持每个目标 ID 保持不变。

Byte 采用一种新的数据关联方法，是整篇论文最关键所在。区别于其他只对高置信度检测框进行数据关联的方法，BYTE 几乎保留所有检测框并进行身份匹配，并将其划分为高置信度和低置信度两组。首先将高置信度检测框关联进轨迹中，然后再将低置信度检测框与未匹配的跟踪对象相关联来保留低置信度检测框并过滤背景。

在新发明的 byte 数据关联基础上，还加入了卡尔曼滤波器和匈牙利匹配算法，形成了一个完整高效的多目标追踪算法。

使用此算法，在本任务中大大提高了战斗车车识别的成功率。

模型性能：

在测试情形下，考察模型 F1-confidence 曲线，如图 37 所示，绝大多数是维系在 1.0（横纵两根轴）附近，有很高的精确度。

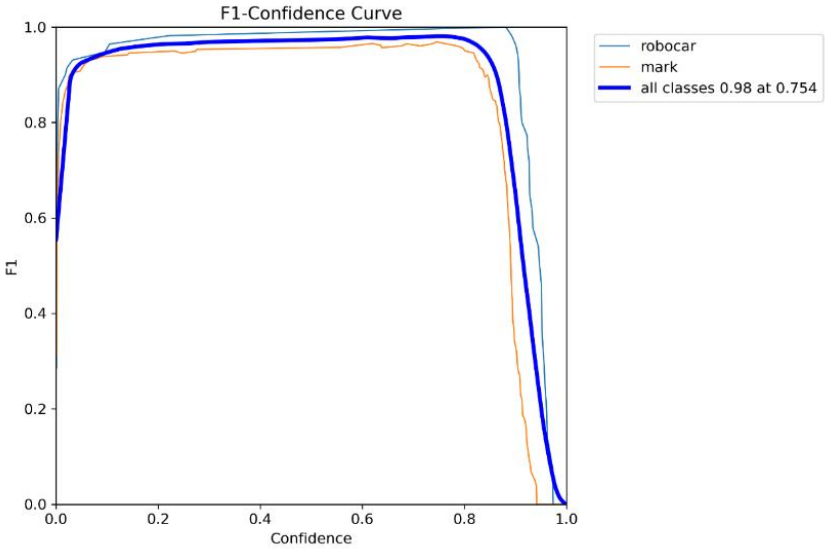

图 37 F1-confidence 指标

各类机器学习指标（loss，precision，recall 等）也表现十分优异，如图 38 所示。

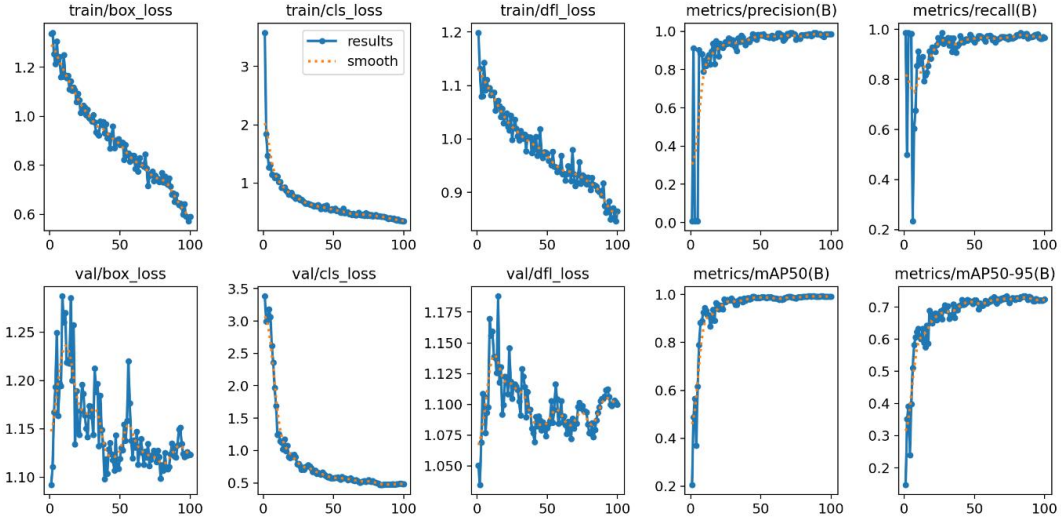

图 38 各项机器学习指标

4.2 Unity 实时战场模拟实况

针对观众群体，本团队专门设计了游客端 App。该 App 由 Unity 开发，并使用了 SolidWorks 建模功能。通过该 App，观众可以实时观看战斗车的比赛。

其中 SolidWorks 建模技术确保了小车模型的高度真实性和准确性，而 Unity 的强大渲染引擎则让观众能够以高品质的视觉体验参与到比赛中。发的

通过这种实时控制和真实建模的融合，为观众带来了全新的观看体验。效果图如图 39 与图 40 所示。

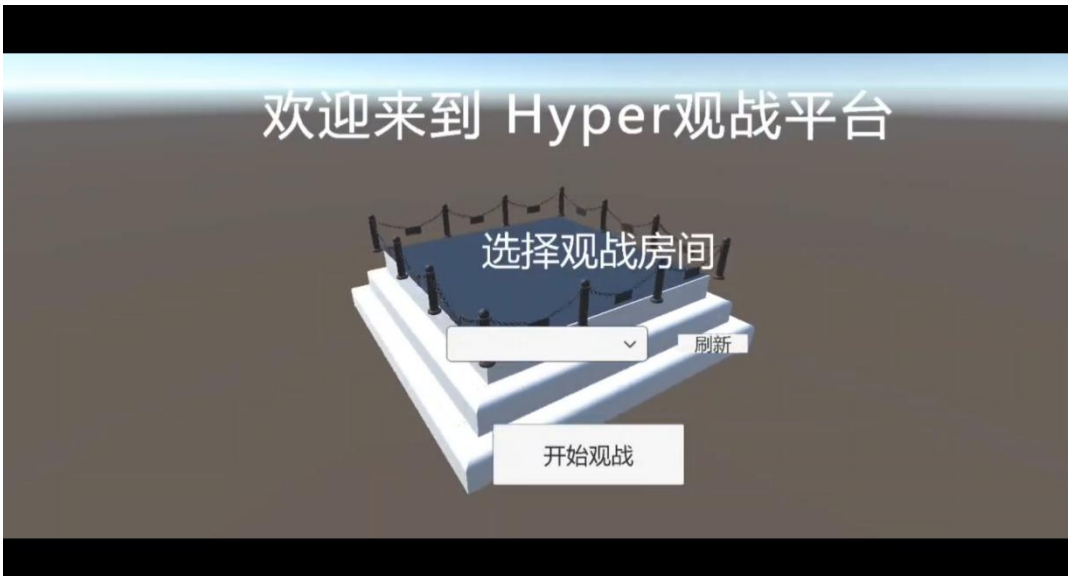

图 39 游客 App 主页

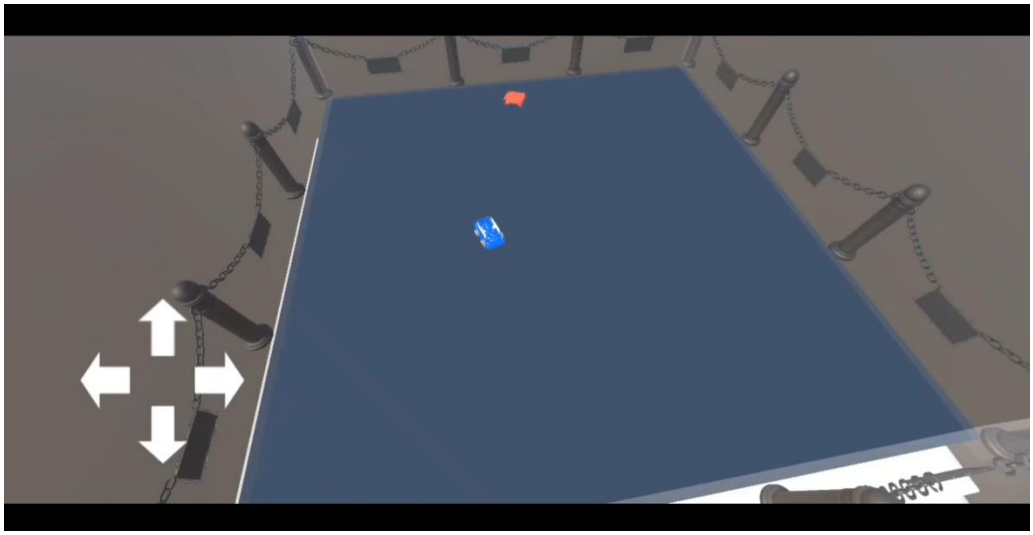

图 40 游戏对战实况展示

4.3 基于 I^2C 协议的可弹出 Hyper 接口

由于传统的 I^2C 总线连接较为紧固不满足可掉落需求，因此为了满足既要主机和模块间交流信息，又要保持接口可弹出，本团队选择使用电磁铁作为连接接口。电磁铁可产生自动吸附的效果，在模块报废的时候也可自由掉落弹出。

Hyper 接口分为两种：

1. 可弹出式：带推拉式电磁铁 + 四电机接口

2. 不可弹出式：仅四电极接口

其中四电极接口是由本团队制作的便于推拉式电磁铁顶出脱落的接口，考虑到当模块失效时连线也要同可拔插模块一同断开，因此制作了此种接口，分为一公一母，便于拔插。本团队使用 I^2C 作为通讯手段，因此其中一对线作为时钟线和数据线，另外一对

则是接地和电源线。

而推拉式电磁铁则是负责将模块弹出，需要和模块直接接触并在检测到模块失效的时候激活使模块自然掉落。

四电极母头如图 41 所示，四电极公头如图 42 所示，四电极公母头相接如图 43 所示，推拉式电磁铁如图 44 所示

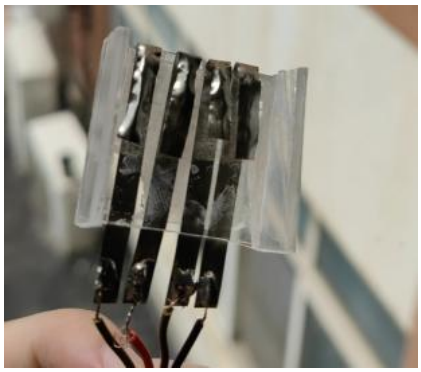

图 41 四电极母头

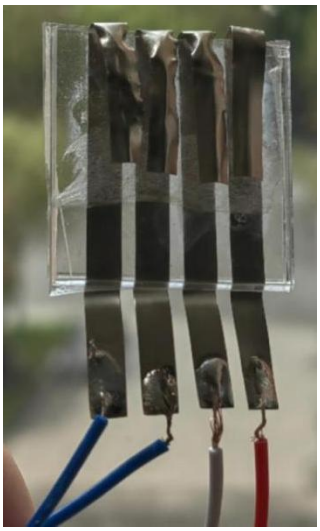

图 42 四电极公头

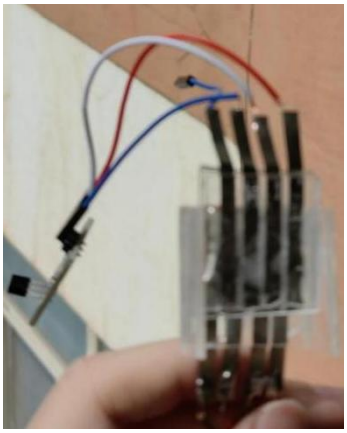

图 43 四电极公母头相接

图 44 推拉式电磁铁

五、组织管理与开发过程

5.1 人员安排

本团队将人员分配为前端 1 人，后端 1 人，硬件 2 人，美工 1 人，如图 45 所示。

其中组长兼任项目经理，主要进行组织管理工作、分配组员各项任务、把控开发进度与进度、协调团队合作等工作。

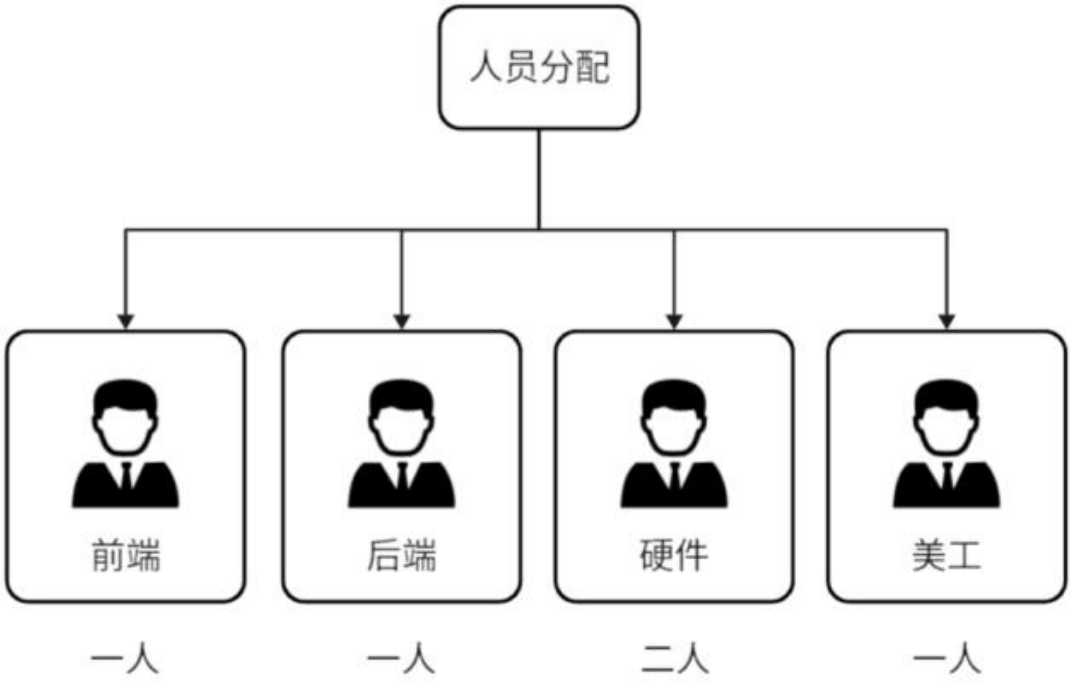

图 45 人员安排

5.2 时间安排

时间安排甘特图如图 46 所示：

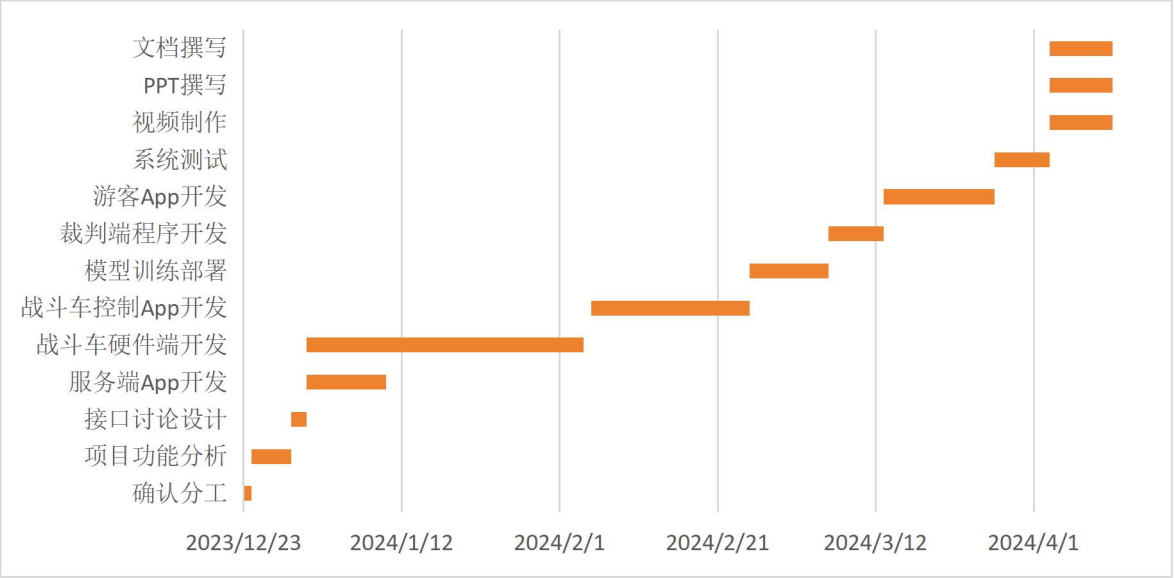

图 46 时间安排甘特图

团队项目计划表如下：

序号	任务内容	开始日期	完成日期	负责人
1	确认分工	2023/12/23	2023/12/24	黄俊儒
2	项目功能分析	2023/12/24	2023/12/29	团队全员
3	接口讨论设计	2023/12/29	2023/12/31	团队全员
4	服务端 App 开发	2023/12/31	2024/1/10	黄俊儒
5	战斗车硬件端开发	2023/12/31	2024/2/5	何浩鹏
6	战斗车控制 App 开发	2024/2/5	2024/2/25	何浩鹏，黄俊儒
7	模型训练部署	2024/2/25	2024/3/6	钱家辉
8	裁判端程序开发	2024/3/6	2024/3/13	黄俊儒，钱家辉
9	游客 App 开发	2024/3/13	2024/3/27	黄圣力

10	系统测试	2024/3/27	2024/4/3	团队全员
11	视频制作	2024/4/3	2024/4/11	钱家辉
12	PPT 撰写	2024/4/3	2024/4/11	巩超，钱家辉
13	文档撰写	2024/4/3	2024/4/11	巩超

六、物料清单与成本证明

6.1 履带式

1. 履带车底盘+电机：199 元
2. 3s 锂电池：47 元
3. 12v 转 5v 转换器：13 元
4. stm32 防御模块 *2：18 元
5. stm32 攻击模块 *1：18 元
6. 树莓派+摄像头：467 元
7. 亚克力板：15 元
8. 红外线发射器（发射模块）*1：9 元
9. 红外线接收器（防御模块）*2：18 元
10. 红外线接收器（核心模块）*1：9 元
11. 平衡充电器：98 元（可供选择）

总计：814 元（不含充电器）、912 元（含充电器）

6.2 四轮式

1. 四轮车底盘+电机：36 元
2. 3s 锂电池：47 元
3. 12v 转 5v 转换器：13 元
4. stm32 防御模块 *2：18 元

5. stm32 攻击模块 *1：18 元

6. 树莓派+摄像头：467 元

7. 亚克力板：15 元

8. 红外线发射器（发射模块）*1：9 元

9. 红外线接收器（防御模块）*2：18 元

10. 红外线接收器（核心模块）*1：9 元

11. 平衡充电器：98 元

总计：650 元（不含充电器）、748 元（含充电器）

成本证明如下所示：

图 47 3s 电池*2

图 48 电压转换器

图 49 stm32 开发板

图 50 履带式行动模块

图 51 平衡充电器

图 52 树莓派 4B 开发板

图 53 四轮式行动模块

图 54 推拉式电磁铁

图 55 亚克力板

图 56 亚克力板 400x400x2

七、项目总结

本项目致力于实现一个可通过扩展不同的攻击、防护等模块，打造出功能各异，可遥控对战的战斗机器人，打造一个可定制，社交性强，应用广泛的新型机器人对战平台。

核心功能——用户 App 控制、战斗车的组装与控制、服务端信息查看、游戏规则的设计。其中用户 App 具体包括：用户登录注册、用户与战斗车的绑定、个人信息查看、房间的创建与加入、游戏结果结算、用户信息查看、排行榜查看；战斗车的模块化组装具体包括：车辆平台选型的设计、外挂模块（核心模块、武器模块、装甲模块、行动模块、外观模块）的设计、战斗车的控制（行走与攻击）、模块挂载识别、重量与速度的转换、战斗车与用户 App 的通信、战斗车第一视角的显示、模块弹出；服务端信息查看方面具体包括：当前在线人数、当前房间数；游戏规则方面本团队设计了多种属性值以及多种模块间的制约关系以及每种模块不同的类型，并设计了电子围栏与出界监测。在拓展方面，本团队提供了游客端 App 以进行线上游戏实况展示。以供本团队希望凭借上述核心功能给用户带来快乐刺激的游戏体验。

本项目也颇具亮点。本团队借助 YOLOv8 与 ByteTrack 模型进行围栏柱识别与战斗车识别，实现了电子围栏的实现与出界监测，节约了玩家游戏成本的同时也帮助游戏能够更有效公平有趣的进行。除此以外，本团队使用拥有强大渲染引擎的 Unity 与 SolidWorks 开发了游客端 App，让观众能够以高品质的视觉体验参与到比赛中。

技术层面上，为在保证项目的开发效率以及产品质量的同时使用前沿 IT 技术，本项目前端采用 Flutter 框架。后端根据场景的不同以及性能的要求在性能敏感部分采用了 Rust 与 Tokio 框架进行高效信息交互，性能不敏感部分采用安全性高且拓展性强的 Django 框架，并针对长期存储与短期低延时响应采用 MySQL 与 Redis 数据库。最后通过 Cordova 将前端界面转换为 Android webview 上的页面，并对接口进行相关适配。硬件采用 Raspberry Pi 4B 主板开发。

团队层面上，团队 5 人分工明确，并保持着良好的沟通与协作，以确保任务分配得当，并及时解决项目中出现的任何问题，且运用创新思维不断地对项目进行优化，从而共同完成了本软件的开发。本团队将继续保持这种协作和团队精神，在未来取得更多的成就。

本项目在规划期内顺利完成，完善了核心功能，并切实考虑用户需求，系统功能全面，界面交互良好。